CS1910 Computer Science I • Decisions

Partner workbook

The Bank Machine and the Free Throw

In lecture with a partner • about 30 minutes

Name: _____ **Partner:** _____

This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.

What this workbook practises

Real-world tasks are full of decisions. You will describe one (using a bank machine) as an algorithm, test it with tricky cases, and then turn a set of basketball rules into nested `if` statements.

1 The bank machine

about 10 minutes

The problem

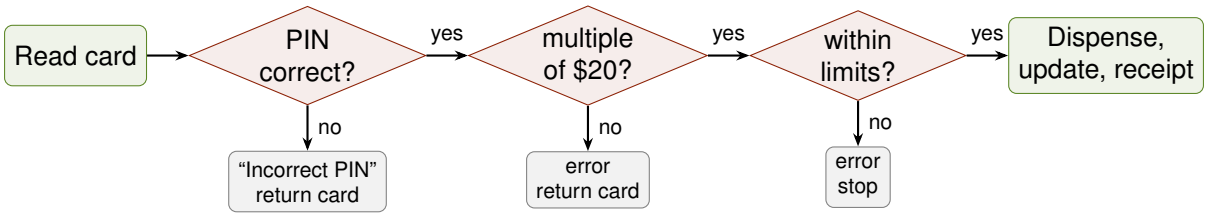
You are going to a bank machine to withdraw some money.

1. What are you trying to achieve?

2. What steps do you take, in order?

3. What might stop you from getting your money?

4. Pick one thing that could stop you, and write it as "If _____ is true, then _____".



One possible bank-machine algorithm as a flowchart. Compare it with your steps.

2 Test your algorithm

about 8 minutes

5. Walk through your steps (or the flowchart) with each case. What should happen? Did your algorithm do the right thing every time?

Case	What should happen?	OK?
Wrong PIN		
Balance $100, request $100		
Balance $100, request $120		
Request $30		
Request $0		

6. Which of these cases is an *edge case*? Why do bugs hide there?

3 Complete the basketball example

about 10 minutes

WNBA shooting fouls

- Fouled while shooting and the shot **misses**: free throws equal to the shot's value (2 for a 2-pointer, 3 for a 3-pointer).
- Fouled and the shot **goes in**: the shot's points **plus 1** free throw.
- Shot misses, no foul: no points. Shot goes in, no foul: the shot's points.

7. Fill in the table of all the cases.

Shot value	Made?	Fouled?	Points	Free throws
2	yes	yes		
2	yes	no		
2	no	yes		
2	no	no		
3	yes	yes		
3	no	yes		

8. Write nested `if` statements that set `points` and `free_throws` from `value` (2 or 3), `made` and `fouled` (both `True` or `False`).

9. Brainstorm another real-world example that would need complex logic or nested `if` statements.

Before you leave

about 1 minutes

- [ ] I can describe a real task as ordered steps with decisions.
- [ ] I can test an algorithm with normal and edge cases.
- [ ] I can write nested `if` statements from a set of rules.